

VETORES E LISTAS

Tiago Henrique Angioletti





VETORES

Vetores são estruturas de dados unidimensionais que armazenam uma coleção de elementos do mesmo tipo.



DECLARAÇÃO E INICIALIZAÇÃO DE VETORES

```
// Declaração e inicialização
int[] vetorInteiros = new int[5]; // Vetor de inteiros com 5 elementos

double[] vetorDecimal = { 1.5, 2.7, 3.0, 4.2 }; // Inicialização direta

vetorInteiros[0] = 10; // Atribuição aos elementos

int primeiroElemento = vetorInteiros[0]; // Acessando o primeiro elemento
```



PERCORRENDO UM VETOR

```
// Percorrendo um vetor usando for
for (int i = 0; i < vetorDecimal.Length; i++)
{
    Console.WriteLine(vetorDecimal[i]);
}
// Percorrendo um vetor usando foreach
foreach (double elemento in vetorDecimal)
{
    Console.WriteLine(elemento);
}
```



MÉTODOS ÚTEIS PARA VETORES

```
// Encontrar o índice de um elemento  
int indice = Array.IndexOf(vetorInteiros, 10);  
  
// Ordenar o vetor  
Array.Sort(vetorInteiros);  
  
// Inverter a ordem do vetor  
Array.Reverse(vetorInteiros);
```



LIST NO C#

No C#, `List` é uma **COLEÇÃO GENÉRICA** (classe da biblioteca `System.Collections.Generic`) usada para armazenar e manipular uma lista de objetos do mesmo tipo.

Diferente de arrays, o tamanho de uma `List` pode crescer ou diminuir dinamicamente conforme você adiciona ou remove elementos.



PRINCIPAIS CARACTERÍSTICAS

GENÉRICA: Você define o tipo de dados que a lista vai conter (ex: `List<int>`, `List<string>`, etc.).

REDIMENSIONÁVEL: Não tem tamanho fixo; pode crescer ou diminuir conforme necessário.

MÉTODOS ÚTEIS: Como `Add()`, `Remove()`, `Find()`, `Sort()`, entre outros.

CRIANDO UMA LIST



```
// Criando uma lista de inteiros
List<int> numeros = new List<int>();

// Adicionando elementos
numeros.Add(10);
numeros.Add(20);
numeros.Add(30);

// Iterando na lista
foreach (int numero in numeros)
{
    Console.WriteLine(numero);
}
```

Saída

```
10
20
30
```




PRINCIPAIS MÉTODOS

- `Add (T item)`: Adiciona um item ao final da lista.
- `Remove (T item)`: Remove a primeira ocorrência do item especificado.
- `RemoveAt (int index)`: Remove o item na posição especificada.
- `Count` : Retorna o número de elementos na lista.
- `Contains (T item)`: Verifica se um item está presente.
- `IndexOf (T item)`: Retorna o índice do item.
- `Clear ()`: Remove todos os elementos da lista.

EXEMPLO COMPLETO



```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> nomes = new List<string> { "Alice", "Bob", "Carlos" };

        // Adicionar
        nomes.Add("Diana");

        // Remover
        nomes.Remove("Bob");

        // Verificar se contém "Carlos"
        bool contemCarlos = nomes.Contains("Carlos");

        // Obter quantidade de elementos
```